

Final Report

담보코드 분석에 따른 보험 추천 알고리즘



강좌	캡스톤 디자인
담당 교수	김일주 교수님
학과	응용소프트웨어 학과
학번(이름)	60160336 황승호 60191434 배정연 60191440 양지선

목 차

1. Topic 소개	3
2. 팀원 역할 소개	3
3. 데이터 소개 및 분석	4
3.1 데이터 소개	4
3.1.1 고객 계약 정보 데이터	4
3.1.2 상품정보(정상외보험) 데이터	5
3.1.3 건강정보 데이터	6
3.1.4 고객 보장 정보 데이터	6
3.2 데이터 분석	7
3.2.1 고객 나이 분포	7
3.2.2 고객 성별 분포	8
3.2.3 보험료 통계	8
3.2.4 담보코드 파악	8
3.2.5 건강위험도 등급	9
4. 모델링을 위한 데이터 전처리 과정	10
4.1 고객 데이터 로드 및 전처리	10
4.1.1 고객 계약 정보 데이터 전처리	10
4.1.2 상품 정보(정상외보험) 데이터에서 고객 정보만 추출하여 전처리	11
4.1.3 고객 계약 정보와 상품 정보 (정상외보험) 데이터의 고객 정보 병합 및 전처리	11
4.2 건강정보 데이터 전처리	12
4.3 담보코드 관련 데이터 전처리	13
4.4 데이터 정규화	13
4.5 PCA (주성분 분석)	14
5. 데이터 모델링 기법 및 모델링 결과	14
5.1 DBSCAN	14
5.1.1 DBSCAN 설명	14
5.1.2 DBSCAN 모델링	15
5.1.3 DBSCAN 모델링 시각화	17

5.1.4 모델링 결과를 바탕으로 데이터를 입력했을 때 예측하기.....	17
5. 2 FUZZY	19
5.2.1 퍼지 모델 설명	19
5.2.2 군집 개수 결정.....	20
5.2.3 퍼지 모델 생성.....	21
5.2.4 군집 할당 및 파악	22
5.2.5 담보코드와 군집 병합	23
5.2.6 담보코드 추천.....	23
5. 3. K-Means.....	24
5.3.1 K-Means Cluster Model 설명	24
5.3.2 Elbow 기법을 이용한 적절한 K 개의 클러스터 개수 결정.....	24
5.3.3 PCA 를 이용하여 클러스터 시각화	25
5.3.4 K-Means 클러스터(k=6) 모델의 군집 별 의미 파악	26
5.3.5 고객 군집과 보장담보코드 간의 User-based CF Pivot Table	27
6. 각 모델링 최종 결과 해석 및 개선점 탐구	29
6. 1 DBSCAN	29
6. 2 FUZZY	29
6. 3 K-Means	30

1. Topic 소개

기존 고객들의 보험 가입 계약은 보험설계사의 주관적인 견해에 따른 추천 또는 보험사의 이익을 극대화해주는 상품들을 위주로 일방적인 가입 계약이 이루어지곤 했다. 또한, 수많은 보험상품 및 담보들과 복잡한 보장내용들에 의해 보험설계사 및 전문가의 도움을 통해 계약이 진행되었다. 이로 인해 고객들은 자신에게 불필요한 보험을 계약하거나 과한 보험료를 지급하게 되어 보험 가입에 소극적으로 임하게 되며, 이는 결국 보험 생태계를 위축시키게 된다.

현 시대의 고객들은 스스로 합리적인 판단을 할 수 있으며, 자신의 상황과 여유에 따라 소비하는 것을 지향한다. 현재 많은 서비스에서 제공되는 추천 시스템은 데이터 기반으로 요청 유저와 유사한 유저들을 찾아내어 그들이 먼저 구매하거나 행동(acting, transaction)한 데이터들을 제공한다. 이를 보험 가입 프로세스에 접목하면, 데이터 기반의 보험 상품 추천을 통해 고객은 합리적인 판단을 할 수 있으며, 자신의 상황 및 여유가 유사한 고객들이 계약한 보험 상품의 추천은 현 시대 고객들의 요구를 충족시킬 수 있다. 이를 통해 고객의 소극적인 보험 가입을 완화하고 보험 생태계를 활성화시킬 수 있을 것으로 기대된다.

본 팀은 아이지넷 사에서 제공하는 보험 마이데이터를 기반으로 보험 보장담보코드 추천 시스템을 구축하고자 한다. 보험은 유사 시 가입된 보험의 보장 내용을 근거로 고객에게 보상금을 지급하는 금융 상품이다. 보험은 여러 보장담보코드를 가지고 있으며, 보장담보코드는 국가에서 지정한 담보 내용에 대한 고유 코드를 말한다. 국내 보험사는 보장담보코드를 기반으로 보험 상품을 설계하고 보험금을 산정하고 있기 때문에, 고객에게 필요한 보장담보코드 위주로 설계된 보험 상품을 추천하는 것은 고객의 불필요한 보험 계약을 지양하도록 기여하며, 자신과 유사한 타 고객들이 많이 가입한 보장담보코드들의 추천은 고객의 합리적인 판단에 기여할 수 있다.

본 팀은 상기 추천 시스템을 다음과 같이 모델링하고자 한다. 고객의 인류통계학적 데이터를 기반으로 고객을 군집화하고, 군집군과 보장담보코드와의 관계에 따른 User-based collaborative filtering 을 설계하여, 각 군집의 인기 보장담보코드를 새로운 고객에게 추천하도록 한다. 이를 위해 사용되는 군집 모델로는 DBSCAN, Fuzzy, K-Means 모델을 선정하였으며, 세 가지 군집 모델로 설계된 User-based CF 에 의해 추천되는 보장담보코드를 비교, 분석하고자 한다.

2. 팀원 역할 소개

	학번 이름	역할	모델링
1	60160336 황승호	모델링, 보고서 작성	K-Means
2	60191434 배정연(팀장)	보고서 초안 작성, 모델링, 보고서 작성, 발표	DBSCAN
3	60191440 양지선	모델링, 보고서 작성, 발표 PPT 제작	Fuzzy

3. 데이터 소개 및 분석

3.1 데이터 소개

이번 캡스톤 프로젝트는 (주)아이자넷에서 제공받은 보험사 데이터를 활용하여 진행하였다. 3 조는 총 4 개의 데이터 파일(고객 계약정보, 상품정보(정상외보험), 건강정보, 고객 보장정보)을 사용했다.

3.1.1 고객 계약 정보 데이터

	id	birthdate	gender	orgName	insuNum	prodName	insuType	insuStatus	issueDate	payAmt	payCycle
0	5443	1973-02-08	여성	메리츠화재해상보험주식회사	615836469	(무) 메리츠 The좋은 알뜰한건강보험 2002	3	2	2020-03-31	31100	1M
1	5443	1973-02-08	여성	하나손해보험	L21C70000141	무배당 하나 Grade 건강보험 2종(세만기형, 해지환급금미지급형II, 건강Grad...	4	2	2021-09-02	50270	1M
2	5443	1973-02-08	여성	농협손해보험 주식회사	3190628998311	(무) NH가성비굿건강보험2001	3	2	2020-01-03	28587	1M
3	158486	1975-02-14	여성	염지손해보험주식회사	20210216883	(무)스마트건강종합보험(21.04)(세만기형, 해지환급금미지급형(납입후50%, 8대납...	3	2	2021-06-14	66243	1M
4	158486	1975-02-14	여성	농협손해보험 주식회사	3190642152921	(무) NH가성비굿건강보험2001	3	2	2020-01-31	21001	1M
...	...	...	...	...	...	...	...	...	...	...	...
9995	182452	1979-07-12	여성	디비손해보험 주식회사	320171663928	참좋은 우리아이보험1704	4	2	2017-12-22	74860	1M
9996	182452	1979-07-12	여성	디비손해보험 주식회사	320171663933	참좋은종합보험1709	3	2	2017-12-22	25000	1M
9997	182457	1988-01-07	남성	메리츠화재해상보험주식회사	608909452	(무) 알파Plus보장보험1701	3	2	2017-01-18	142900	1M
9998	182456	1984-08-16	여성	디비손해보험 주식회사	320160452580	내인생행복플러스종합보험1601	3	2	2016-03-30	129850	1M
9999	182456	1984-08-16	여성	디비손해보험 주식회사	320161013241	우리아이첫보험1601	4	2	2016-07-15	30000	1M

고객 계약정보에는 총 10000 개의 고객 계약 데이터가 담겨있으며, 총 11 개의 컬럼으로 이루어져 있다.

컬럼명	의미
id	고객의 id
birthdate	고객의 생년월일
gender	고객의 성별
orgName	보험회사명
insuNum	증권번호
prodName	보험상품명
insuType	보험종류
insuStatus	주계약 상태
issueDate	보험가입일
payAmt	월 보험료
payCycle	납입 주기

3.1.2 상품정보(정상외보험) 데이터

상품정보(정상외보험)에는 총 10000 개의 고객 계약 데이터가 담겨있으며, 총 11 개의 컬럼으로 이루어져 있다.

	id	birthdate	gender	orgName	insuNum	prodName	insuType	insuStatus	issueDate	payAmt	payCycle
0	131232	1987-02-24	남성	현대해상화재보험	L0131284651400000	무배당멀티플암보험(Hi1304) 기본플랜	5	4	2013-05-28 0.00	40000	1M
1	75695	1970-08-15	남성	DB생명보험주식회사	960857102	새가족보장보험(2월,60세만기,개인형)	2	4	1996-08-30 0.00	56700	1M
2	156852	1981-08-02	남성	메리츠화재해상보험 주식회사	61026632	(무) 메리츠 걱정없는 암보험 1801(1종)	5	4	2018-01-12 0.00	32000	1M
3	156852	1981-08-02	남성	디비손해보험 주식회사	3.2018E+11	참좋은암보험1802	5	4	2018-02-05 0.00	50000	1M
4	10486	1979-02-10	남성	메리츠화재해상보험 주식회사	6089645251	(무) 메리츠 The알뜰한건강보험1701	3	4	2017-06-29 0.00	100620	1M
...	...	...	...	...	...	...	...	...	...	...	...
9995	239234	1991-08-24	남성	에이아이에이생명보험 주식회사	64995217	무) 이좋은치아	3	4	2018-09-14 0.00	17379	1M
9996	73795	1983-08-29	남성	KB라이프생명보험	2010265591	무) 소득보장보험	2	4	2017-07-15 0.00	10500	1M
9997	73795	1983-08-29	남성	KB라이프생명보험	2010305534	무) 소득보장보험	2	4	2020-06-25 0.00	9900	1M
9998	73795	1983-08-29	남성	KB라이프생명보험	2010305542	무) 소득보장보험	2	4	2020-06-25 0.00	9900	1M
9999	239287	1950-06-02	여성	열지손해보험주식회사	20165305422	무배당 한지봉 상해종합보험 (1611)(1종)(주택플랜)	4	6	2016-11-09 0.00	30000	1M

컬럼명	의미
id	고객의 id
birthdate	고객의 생년월일
gender	고객의 성별
orgName	보험 회사명
insuNum	증권번호
prodName	보험 상품명
insuType	보험종류
insuStatus	주계약 상태
issueDate	보험 가입일
payAmt	월 보험료
payCycle	납입 주기

3.1.3 건강정보 데이터

	customer_id	type	incidence	rate	cancer_10_year
0	75695	blca	-19.067603	1.0	13.8
1	75695	brca	NaN	NaN	NaN
2	75695	coca	-47.737596	1.0	65.1
3	75695	gaca	-53.499030	1.0	77.9
4	75695	lica	-82.169023	1.0	45.0
...	...	...	...	...	...
11515	134163	ovca	34.411980	4.0	10.5
11516	134163	paca	76.655652	5.0	12.9
11517	134163	prca	NaN	NaN	NaN
11518	134163	thca	18.511272	3.0	78.5
11519	134163	utca	80.571498	5.0	11.6

건강정보에는 11520 개의 데이터가 담겨있으며, 총 5 개의 컬럼으로 이루어져있다.

컬럼명	의미
customer_id	고객의 id
type	암 종류
incidence	암 발생 수치
rate	위험도 등급
cancer_10_year	최근 10 년간 발생률

- type: blca(방광암), brca(유방암), coca(대장암), gaca(위암), lica(간암), ovca(난소암), paca(췌장암), prca(전립선암), thca(갑상선암), utca(자궁암)

3.1.4 고객 보장 정보 데이터

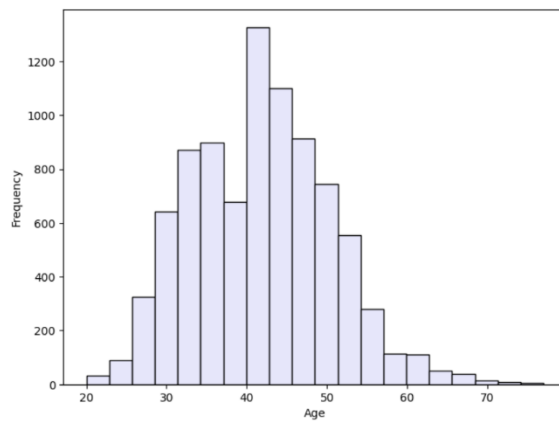
	id	성별	연령	insuNum	prodName	status	coverageCode	type	coverageNum	coverageName	coverageAmt
0	182457	NaN	NaN	608909452	(무) 알파Plus보장보험 1701	1	A5999	NaN	630540	관절염,생식기질환수술비	500.0
1	182457	NaN	NaN	608909452	(무) 알파Plus보장보험 1701	1	A5999	NaN	630542	치핵수술비	200.0
2	182457	NaN	NaN	608909452	(무) 알파Plus보장보험 1701	1	A4199	NaN	630638	결핵 진단비	300.0
3	182457	NaN	NaN	608909452	(무) 알파Plus보장보험 1701	1	A4199	NaN	630639	악제내성결핵(슈퍼결핵 포함) 진단비	10000.0
4	182457	NaN	NaN	608909452	(무) 알파Plus보장보험 1701	1	A5199	NaN	630651	30대질 병수술비	3000.0
...	...	...	...	...	...	...	...	...	...	...	...
119813	28	NaN	NaN	5.63899E+12	無여성시대건강보험(환 급)	1	A5200	NaN	LI0114002_W0085_001	암치료	5000.0
119814	28	NaN	NaN	5.63899E+12	無여성시대건강보험(환 급)	1	A6200	NaN	LI0114002_W0108_001	암치료	40.0
119815	28	NaN	NaN	5.63899E+12	無여성시대건강보험(환 급)	1	A9999	NaN	LI0114002_W0140_001	암치료	2000.0
119816	28	NaN	NaN	5.63899E+12	無여성시대건강보험(환 급)	1	A6100	NaN	LI0041002_W0094_001	無입원	10.0
119817	28	NaN	NaN	5.63899E+12	無여성시대건강보험(환 급)	1	A6300	NaN	LI0041002_W0109_001	無입원	10.0

고객 보장정보에는 119818 개의 데이터가 담겨있으며, 총 11 개의 컬럼으로 이루어져 있다.

컬럼명	의미
id	고객의 id
성별	고객의 성별
연령	고객의 연령
insuNum	증권번호
prodName	보험 상품명
status	주계약 상태
coverageCode	담보코드
type	담보특성
coverageNum	담보일련번호
coverageName	회사담보명
coverageAmt	보장금액

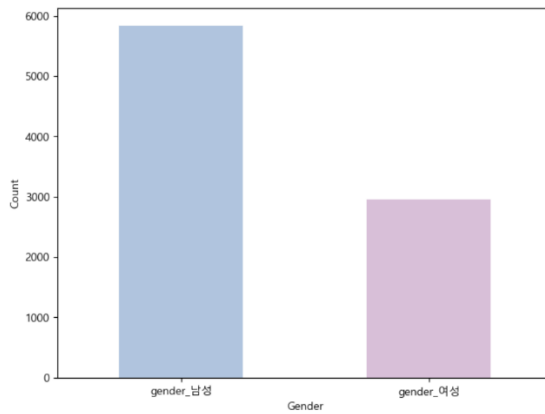
3.2 데이터 분석

3.2.1 고객 나이 분포



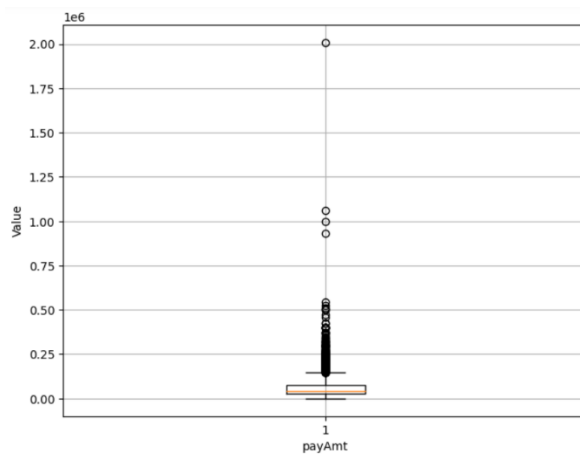
고객 데이터 중 나이는 주로 40-50 세에 많이 분포하는 것을 알 수 있다.

3.2.2 고객 성별 분포



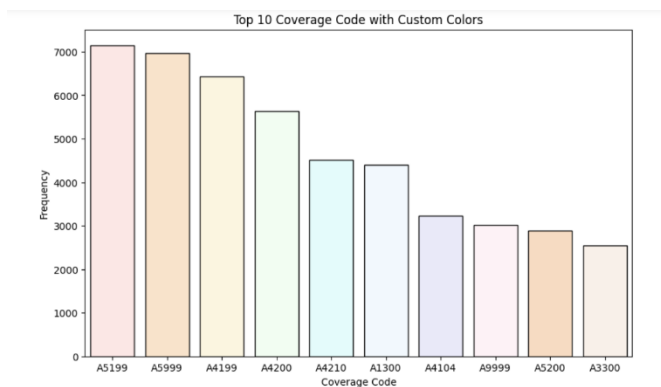
고객 데이터 중 성별은 남성이 약 2 배 정도 더 많이 분포한 것을 알 수 있다.

3.2.3 보험료 통계



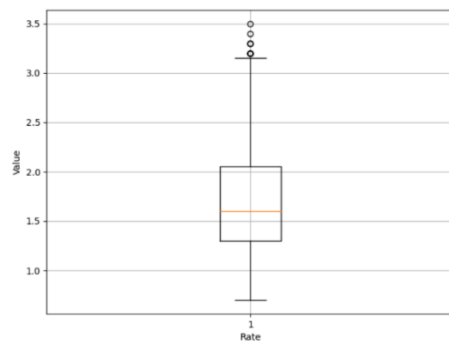
월 보험료의 통계수치 (평균: 57,473.36 원, 최솟값: 0 원, 제 1 분위수: 28,457 원, 중위값: 44,476.10 원, 제 3 분위수: 71,721.25 원, 최댓값: 2,008,700 원) 를 파악했다.

3.2.4 담보코드 파악



담보코드의 범주는 총 185 개이며, 데이터는 이 중 A5199, A5999, A4199 순으로 가장 많은 것을 알 수 있다.

3.2.5 건강위험도 등급



건강정보 데이터 중 고객별 평균 건강 위험도는 주로 1.3 ~ 2.05 에 많이 분포, 일부 데이터 3.0 이상임을 알 수 있다.

4. 모델링을 위한 데이터 전처리 과정

4.1 고객 데이터 로드 및 전처리

4.1.1 고객 계약 정보 데이터 전처리

```
df_1 = pd.read_excel('data/01.Data/1차Data/고객 계약정보Data(Sample10,000).xlsx')

# 사용 컬럼 (id, age, gender, birthdate, payAmt)
# 불필요한 컬럼 삭제
df_1.drop(columns=['orgName', 'insuNum', 'prodName', 'insuType', 'insuStatus', 'issueDate', 'payCycle'], inplace=True)

# id별 평균 월보험료로 그룹화
df_1 = df_1.groupby(['id', 'birthdate', 'gender'])['payAmt'].mean().reset_index()

# 생년월일 변환
df_1['birthdate'] = pd.to_datetime(df_1['birthdate'])

# 현재 날짜 계산
current_date = datetime.now()

# 만 나이 계산 함수 정의
def calculate_age(birthdate, current_date):
    age = current_date.year - birthdate.year - ((current_date.month, current_date.day) < (birthdate.month, birthdate.day))
    return age

# 'age' 컬럼에 만 나이 추가
df_1['age'] = df_1['birthdate'].apply(lambda x: calculate_age(x, current_date))
df_1.drop(columns=['birthdate'], inplace=True)

# gender 컬럼의 남성/여성을 원핫인코딩 기법으로 전처리
df_1 = pd.get_dummies(df_1, columns=['gender'])

df_1.head(5)
```

	id	payAmt	age	gender_남성	gender_여성
0	28	0.000000	44	False	True
1	76	48024.000000	48	False	True
2	87	0.000000	44	False	True
3	113	76225.000000	29	True	False
4	117	26723.333333	46	True	False

고객의 군집화를 위해서는 고객 데이터가 필요하기에 고객 계약 정보 데이터 고객 데이터를 추출하여 전처리하였다. 전처리 과정은 고객 계약 정보 데이터 중 사용하지 않을 데이터는 제거하고 월보험료는 평균 보험료로 계산되도록 그룹화하고 현재 날짜를 기반으로 생년월일을 나이로 변환하여 전처리 하였다.

4.1.2 상품 정보(정상외보험) 데이터에서 고객 정보만 추출하여 전처리

```
df_2 = pd.read_excel('data/01.Data/2차Data/상품정보_정상외보험_20231031.xlsx', skiprows=1)

# 사용 컬럼 (id, age, gender, birthdate, payAmt)
# 불필요한 컬럼 삭제
df_2.drop(columns=['orgName', 'insuNum', 'prodName', 'insuType', 'insuStatus', 'issueDate', 'payCycle', 'Unnamed: 11', 'Unnamed: 12'], inplace=True)

# id별 평균 월보험료로 그룹화
df_2 = df_2.groupby(['id', 'birthdate', 'gender'])['payAmt'].mean().reset_index()

# 생년월일 변환
df_2['birthdate'] = pd.to_datetime(df_2['birthdate'])

# 현재 날짜 계산
current_date = datetime.now()

# 만 나이 계산 함수 정의
def calculate_age(birthdate, current_date):
    age = current_date.year - birthdate.year - ((current_date.month, current_date.day) < (birthdate.month, birthdate.day))
    return age

# 'age' 컬럼에 만 나이 추가
df_2['age'] = df_2['birthdate'].apply(lambda x: calculate_age(x, current_date))
df_2.drop(columns=['birthdate'], inplace=True)

# gender 컬럼의 남성/여성을 원핫인코딩 기법으로 전처리
df_2 = pd.get_dummies(df_2, columns=['gender'])

df_2.head(5)
```

	id	payAmt	age	gender_남성	gender_여성
0	70	30752.333333	41	False	True
1	90	58440.000000	53	True	False
2	150	15793.333333	43	True	False
3	418	119574.000000	40	True	False
4	429	25701.500000	30	True	False

데이터를 살펴보니 상품 정보(정상외보험) 데이터의 고객과 앞서 전처리한 고객 계약정보 데이터의 고객이 서로 달라서 고객 정보가 많을수록 데이터의 모델링이 정확하기에 정상외보험 데이터의 정보도 고객 계약 정보 전처리와 동일한 방법으로 진행하였다.

4.1.3 고객 계약 정보와 상품 정보 (정상외보험) 데이터의 고객 정보 병합 및 전처리

```
df = pd.concat([df_1, df_2])
df = df.drop_duplicates(subset='id', keep='first')

df['gender_남성'] = df['gender_남성'].replace({True:1, False:0})
df['gender_여성'] = df['gender_여성'].replace({True:1, False:0})

df.head(5)
```

	id	age	gender_남성	gender_여성
0	5443	50	0	1
3	158486	48	0	1
5	148906	53	0	1
6	158409	30	0	1
7	76233	46	1	0

고객 계약 정보 데이터의 고객 데이터와 상품 정보 정상외보험의 고객 데이터를 병합하고 id 를 기준으로 중복되는 id 를 삭제한다. 그리고 여성과 남성을 0 과 1 로 원핫 인코딩한다.

4.2 건강정보 데이터 전처리

```
df_3 = pd.read_excel('data/01.Data/3차Data/건강정보_20231107.xlsx', skiprows=1)
df_3 = df_3.rename(columns={'customer_id': 'id'})

# 불필요한 컬럼 삭제
df_3.drop(columns=['incidence', 'cancer_10_year'], inplace=True)

# customer_id와 type의 중복 데이터 삭제
df_3 = df_3.drop_duplicates(subset=['id', 'type'])

# rate 결측치 0으로 대체
df_3['rate'].fillna(0, inplace=True)

# 고객별 평균 rate 계산
rate_average = df_3.groupby('id')['rate'].mean().reset_index()
rate_average.describe()
```

	id	rate
count	545.000000	545.000000
mean	127479.906422	1.735596
std	56475.766422	0.643476
min	97.000000	0.700000
25%	83255.000000	1.300000
50%	149972.000000	1.600000
75%	173237.000000	2.000000
max	193335.000000	3.500000

Excel 파일을 읽어서 데이터프레임 df_3 로 저장한다. skiprows=1 은 파일의 첫 번째 행을 건너뛴다는 것을 의미한다. 첫번째 행은 정보가 아니라 구분하는 행이므로 해당 작업을 수행하였다. 그 후 편의 상 열의 이름을 id 로 변경한다. 그 후 쓰지 않는 incidence 와 cancer_10_year 열을 데이터프레임에서 제거한다. 그 후 id 와 type 을 기준으로 중복된 행을 제거한다. 그 후 열에서 누락된 값들을 0 으로 채우고 id 를 기준으로 그룹화한 후, 각 그룹의 rate 평균을 계산하고, 인덱스를 재설정한다. 마지막으로 잘 처리되었는 지 확인하기 위해 rate_average.describe()를 통해 rate 의 평균값들에 대한 기술통계를 출력하여 확인한다.

```
df = pd.merge(df, rate_average, how='left', on='id').fillna(0)
df.head(5)
```

	id	age	gender_남성	gender_여성	rate
0	5443	50	0	1	0.8
1	158486	48	0	1	0.8
2	148906	53	0	1	0.0
3	158409	30	0	1	0.0
4	76233	46	1	0	0.0

데이터프레임을 'id' 열을 기준으로 왼쪽 병합(left join) 한다. 이때 누락된 값은 0 으로 채운다. 병합된 데이터프레임의 상위 5 개 행을 출력하여 병합된 걸 확인한다.

4.3 담보코드 관련 데이터 전처리

```
df_4 = pd.read_excel('data/01.Data/1차Data/고객 보장정보Data(Sample10,000).xlsx')
df_4.drop(columns=['성별', '연령', 'insuNum', 'prodName', 'status', 'type', 'coverageNum', 'coverageName', 'coverageAmt'], inplace=True)
df_4['coverageCode'] = df_4['coverageCode'].dropna()
df_4 = df_4[df_4['coverageCode'].notnull() & df_4['coverageCode'].str.startswith('A')]
df_4.head(5)
```

	id	coverageCode
0	182457	A5999
1	182457	A5999
2	182457	A4199
3	182457	A4199
4	182457	A5199

Excel 파일을 읽어서 데이터프레임 df_4 로 로드한다. df_4.drop(columns=[...], inplace=True)를 통해 데이터프레임에서 필요 없는 여러 열들을 제거한다. df_4['coverageCode'] = df_4['coverageCode'].dropna()를 통해 'coverageCode' 열에서 결측치를 제거한다. df_4 = df_4[df_4['coverageCode'].notnull() & df_4['coverageCode'].str.startswith('A')]를 통해 'coverageCode' 열에서 결측치가 아니고('notnull'), 'A'로 시작하는('startswith') 데이터만 필터링하여 df_4 에 다시 저장한다. df_4.head(5)를 통해 최종적으로 수정된 데이터프레임의 상위 5 개 행을 출력하여 잘 전처리 되었는지 확인한다.

4.4 데이터 정규화

```
# 컬럼 선택 ('id' 제거)
selected_cols = df.columns.drop(['id'])
X = df[selected_cols]

# StandardScaler로 스케일링 수행
X_scaler = StandardScaler().fit(X)
X_scaled = X_scaler.transform(X)
```

selected_cols = df.columns.drop(['id'])를 통해 데이터프레임 df 에서 'id' 열을 제외한 모든 열을 선택한다. 'id' 열은 모델링에 사용되지 않을 식별자로 간주되어 제외된다.

X = df[selected_cols]를 통해 'id'를 제외한 나머지 열들만 포함하는 새로운 데이터프레임 X를 생성한다. 이 X는 머신러닝 모델의 특성(입력 변수)으로 사용된다.

X_scaler = StandardScaler().fit(X)에서는 StandardScaler를 사용하여 X 데이터에 대해 스케일링을 위한 평균과 표준 편차를 계산한다. 이 스케일러는 데이터를 정규화하여 각 특성의 평균을 0, 표준 편차를 1로 만든다.

X_scaled = X_scaler.transform(X): 계산된 평균과 표준 편차를 사용하여 X 데이터를 스케일링한다. 이렇게 변환된 X_scaled는 이제 특성들이 같은 스케일을 가지게 되어 머신러닝 알고리즘에서 더 효과적으로 작동할 수 있다.

4.5 PCA (주성분 분석)

PCA 는 고차원 데이터셋에 있는 중요한 정보를 유지하면서 차원을 줄이는 방법으로, 데이터의 시각화, 노이즈 제거, 효율적인 저장 및 계산을 위해 널리 사용된다.

```
# PCA
pca = PCA(n_components=2).fit(X_scaled)
df_pca = pca.transform(X_scaled)
pca_cols = ['pc1', 'pc2']
df_pca = pd.DataFrame(data=df_pca, columns=pca_cols)
```

`pca = PCA(n_components=2).fit(X_scaled)`를 통해 스케일링된 데이터 `X_scaled`에 대해 PCA를 수행한다. `n_components=2`는 데이터를 가장 잘 대표하는 주성분을 두 개만 유지하겠다는 의미이다. 여기서 `.fit()` 메서드는 이 주성분들을 찾아내기 위한 학습 과정을 수행한다.

`df_pca = pca.transform(X_scaled)`를 통해 앞서 학습된 PCA 모델을 사용하여 스케일링된 데이터를 변환(transform)한다. 이 변환 과정을 통해 원래 데이터셋의 차원을 축소하여 두 개의 주성분으로 이루어진 데이터셋으로 변환한다.

`pca_cols = ['pc1', 'pc2']`를 통해 축소된 데이터셋의 열 이름을 'pc1', 'pc2'로 지정합니다. 여기서 'pc1'은 첫 번째 주성분, 'pc2'는 두 번째 주성분을 의미한다

`df_pca = pd.DataFrame(data=df_pca, columns=pca_cols)`를 통해 변환된 데이터와 열 이름을 사용하여 새로운 pandas 데이터프레임 `df_pca`를 생성한다. 이 데이터프레임은 이제 두 개의 열, 즉 두 개의 주성분을 가지고 있다.

5. 데이터 모델링 기법 및 모델링 결과

5. 1 DBSCAN

5.1.1 DBSCAN 설명

DBSCAN은 데이터 포인트들 사이의 밀집도에 기초하여 클러스터를 형성하는 머신러닝 알고리즘이다. DBSCAN의 이름은 'Density-Based Spatial Clustering of Applications with Noise'의 약자로, '밀도 기반 공간 클러스터링 및 잡음 응용'이라는 뜻을 가지고 있습니다. 이 알고리즘은 데이터에서 복잡한 구조를 가진 클러스터를 찾을 수 있으며, 클러스터의 형태가 불규칙하거나 클러스터의 크기가 다를 때 특히 유용하다.

DBSCAN 은 다음과 같은 방식으로 동작한다.

핵심 포인트(Core Point) 식별: 알고리즘은 각 데이터 포인트에서 일정 반경(eps) 내에 몇 개의 다른 포인트(min_samples)가 있는지를 확인한다. 이 반경 내에 충분한 수의 포인트가 있다면, 해당 포인트는 핵심 포인트로 간주된다.

클러스터 형성: 핵심 포인트들은 서로 연결되어 클러스터를 형성한다. 만약 핵심 포인트가 서로의 반경 내에 위치한다면, 그들은 동일한 클러스터에 속한다.

경계 포인트(Border Point) 식별: 어떤 포인트가 핵심 포인트의 반경 내에는 있지만, 자신은 핵심 포인트가 아닌 경우, 그 포인트는 경계 포인트로 분류된다. 경계 포인트는 클러스터에 포함되지만, 클러스터를 확장하지는 않는다.

잡음 포인트(Noise Point) 식별: 만약 어떤 포인트가 핵심 포인트도 아니고, 경계 포인트의 반경 내에도 있지 않다면, 그 포인트는 잡음 포인트로 간주되어 어느 클러스터에도 속하지 않는다.

DBSCAN 의 주요 장점은 클러스터의 수를 미리 정하지 않아도 되고, 클러스터의 형태가 원형이 아니어도 동작한다는 것이다. 또한, 이상치(Outlier) 데이터를 잡음으로 구분하여 무시할 수 있기 때문에, 이상치에 강한 클러스터링을 수행할 수 있다.

5.1.2 DBSCAN 모델링

```
# DBSCAN 클러스터링
dbscan = DBSCAN(eps=1, min_samples=5)
clusters = dbscan.fit_predict(scaled_features)

# 클러스터 결과 추가
df['Cluster'] = clusters

# 담보 코드 데이터 로드 및 결합
df = df.merge(df_4, on='id')

# 클러스터별 선호 담보 코드 분석
preferred_collateral = df.groupby('Cluster')['coverageCode'].value_counts().groupby(level=0).head(5)
print(preferred_collateral)
```



```

Cluster  coverageCode
-1      A5999          17
        A4199          11
        A4200           9
        A1300           7
        A5199           7
  0      A5999        2786
        A5199        2547
        A4199        2545
        A4200        1884
        A4210        1489
  1      A5199        4063
        A5999        3717
        A4199        3429
        A4200        3287
        A4210        2640
  2      A5199         336
        A4200         305
        A5999         302
        A4199         301
        A4210         275
  3      A5199         196
        A5999         149
        A4199         148
        A4200         141
        A1300         133
  4      A5104           2
        A5107           2
        A1100           1
        A1200           1
        A1300           1
Name: count, dtype: int64

```

DBSCAN 알고리즘을 사용하여 표준화된 특성 데이터에 대해 클러스터링을 수행한다. `eps=1` 은 클러스터를 형성하기 위한 최대 거리를 의미하고, `min_samples=5` 는 클러스터를 형성하기 위해 필요한 최소 샘플 수를 나타낸다. DBSCAN 알고리즘으로부터 얻은 클러스터 결과를 원래 데이터프레임 `df` 에 'Cluster' 열로 추가한다. `df.merge(df_4, on='id')` 를 통해 담보 코드 데이터(`df_4`)를 기존 데이터프레임 `df` 에 병합한다. 이 과정은 'id'를 기준으로 두 데이터프레임을 연결한다. `df.groupby('Cluster')['coverageCode'].value_counts()`를 이용하여 각 클러스터별로 가장 많이 나타나는 담보 코드를 집계한다. `groupby(level=0).head(5)`를 통해 각 클러스터별로 상위 5 개의 담보 코드를 선택하고 `print(preferred_collateral)`를 통해 이 분석 결과를 출력한다.

[PRINT 결과 해석]

이 표는 DBSCAN 클러스터링 알고리즘을 통해 생성된 클러스터와 각 클러스터 내에서 관찰된 'coverageCode'(담보 코드)의 빈도수를 보여준다. 각 클러스터는 숫자로 표시되며, '-1'은 클러스터에 속하지 않은 잡음 포인트를 나타낸다.

클러스터 -1: 이 클러스터는 실제로 클러스터가 아니며 잡음 포인트들의 모임이다. 가장 많이 나타난 담보 코드는 'A5999'로 17 회 관찰되었고, 그 다음은 'A4199'으로 11 회, 'A4200'으로 9 회 등이다.

클러스터 0: 가장 많이 나타난 담보 코드는 'A5999'로 2786 회, 'A5199'로 2547 회, 'A4199'로 2545 회 등이다.

클러스터 1: 'A5199' 코드가 4063 회로 가장 많이 나타났고, 'A5999'가 3717 회, 'A4199'가 3429 회로 뒤를 이었다.

클러스터 2, 3: 이 클러스터들도 비슷한 패턴을 보이며, 몇몇 담보 코드들이 주로 나타난다.

클러스터 4: 매우 작은 클러스터로 보이며, 'A5104'와 'A5107'이 각각 2 회씩, 나머지 코드들은 1 회씩 나타난다.

5.1.3 DBSCAN 모델링 시각화

```
# preferred_collateral 결과를 시각화하기 위한 데이터 준비
visualization_data = preferred_collateral.reset_index()
visualization_data.columns = ['Cluster', 'coverageCode', 'Count']

# 막대 그래프로 시각화
plt.figure(figsize=(15, 10))

# 각 클러스터별로 서로 다른 색상을 사용하여 막대 그래프를 그림.
sns.barplot(x='Cluster', y='Count', hue='coverageCode', data=visualization_data)

# 로그 스케일로 y-축 설정 ( COUNT의 차이가 극단적이라서 로그 스케일로 결정)
plt.yscale('log')

# 군집별로 다른 색상을 사용하여 막대 그래프를 그림.
sns.barplot(x='Cluster', y='Count', hue='coverageCode', data=visualization_data)

plt.xlabel('Cluster')
plt.ylabel('Count (log scale)')
plt.title('Top 5 Preferred Collateral Codes per Cluster (Log Scale)')
plt.legend(title='Coverage Code')
plt.show()
```

preferred_collateral.reset_index()를 통해 클러스터링 결과 데이터의 인덱스를 초기화하고 데이터프레임으로 변환한다. 새 데이터프레임의 열 이름을 'Cluster', 'coverageCode', 'Count'로 설정한다. plt.figure(figsize=(15, 10))를 사용해 그래프의 크기를 설정한다. sns.barplot() 함수를 사용하여 'Cluster'를 x 축에, 'Count'를 y 축에 두고, 'coverageCode'를 기준으로 다른 색상을 가진 막대 그래프를 그린다. plt.yscale('log')를 통해 y 축을 로그 스케일로 설정한다. 이는 클러스터 내 카운트 값의 큰 범위를 더 잘 표현하기 위한 것이다. x 축에 'Cluster', y 축에 'Count (log scale)', 그래프의 제목으로 'Top 5 Preferred Collateral Codes per Cluster (Log Scale)'을 설정하고, 범례의 제목으로 'Coverage Code'를 설정한다. plt.show() 함수를 호출하여 준비된 막대 그래프를 표시한다.

5.1.4 모델링 결과를 바탕으로 데이터를 입력했을 때 예측하기

```
# 고객 정보 입력
age = int(input("고객의 나이를 입력하세요: "))
gender = input("고객의 성별을 입력하세요 (남성은 'M', 여성은 'F'): ")
avg_rate = float(input("고객의 평균 건강위험도를 입력하세요: "))
avg_payAmt = float(input("고객의 평균 월보현료를 입력하세요: "))

# 성별 원-핫 인코딩
if gender == 'M':
    gender_M = 1
    gender_F = 0
elif gender == 'F':
    gender_M = 0
    gender_F = 1

# 입력된 고객 데이터 정규화
new_customer = pd.DataFrame({'age': age, 'gender_남성': gender_M, 'gender_여성': gender_F, 'rate': avg_rate, 'payAmt': avg_payAmt})

# 클러스터 예측
predicted_cluster = dbscan.fit_predict(new_customer)

# 선호하는 담보 코드 출력
preferred_collateral = df.groupby('Cluster')['coverageCode'].value_counts().groupby(level=0).head(5)
print('선택된 군집의 선호 담보 코드: %n', preferred_collateral[predicted_cluster[0]])
```

```

고객의 나이를 입력하세요: 25
고객의 성별을 입력하세요 (남성은 'M', 여성은 'F'): F
고객의 평균 건강위험도를 입력하세요: 1.0
고객의 평균 월보험료를 입력하세요: 10000
선택된 군집의 선호 담보 코드:
coverageCode
A5999    17
A4199    11
A4200     9
A1300     7
A5199     7
Name: count, dtype: int64

```

이 코드는 고객 정보를 입력받아 해당 고객이 속할 것으로 예상되는 클러스터를 찾고, 그 클러스터에서 선호하는 담보 코드를 예측하는 과정이다. 각 과정을 상세히 설명하자면 다음과 같다.

1. 고객 정보 입력

사용자로부터 고객의 나이, 성별, 평균 건강 위험도, 평균 월 보험료를 입력받는다. 성별을 'M' 또는 'F'로 입력받아 남성의 경우 'M', 여성의 경우 'F'로 구분한다.

2. 성별 원-핫 인코딩

입력받은 성별 정보를 숫자로 변환하기 위해 원-핫 인코딩을 사용한다. 남성일 경우 'gender_male'은 1로, 'gender_female'은 0으로 설정한다. 여성일 경우 'gender_male'은 0으로, 'gender_female'은 1로 설정한다.

3. 고객 데이터 정규화

입력받은 고객 데이터를 데이터프레임 형태로 만들고, 이를 정규화한다. 정규화는 아직 수행되지 않았습디만, 이러한 과정을 거칠 것으로 예상된다.

4. 클러스터 예측

DBSCAN 알고리즘을 사용하여 새로 입력된 고객 데이터에 대한 클러스터를 예측한다.

5. 선호 담보 코드 출력

원래 데이터프레임 **df**에서 클러스터별로 가장 많이 나타나는 담보 코드를 추출한다. 예측된 클러스터 번호에 해당하는 선호 담보 코드를 출력한다.

5. 2 FUZZY

5.2.1 퍼지 모델 설명

퍼지 모델(Fuzzy Model)은 불확실성을 다루기 위해 퍼지 논리와 관련된 개념을 사용하여 모델링하는 방법을 의미한다. 퍼지 논리는 명확한 참/거짓이 아니라 어느 정도의 참임을 나타내는 데에 중점을 둔 논리 체계로, 현실 세계의 불확실하거나 모호한 정보를 다룰 때 특히 유용하다.

퍼지 C-Means 군집화는 다양한 상황에서 장점을 가진다. 퍼지 군집의 가장 큰 장점은 각 데이터 포인트의 군집 소속도를 나타낼 수 있다는 것으로, 각 데이터 포인트가 각 군집에 속할 확률을 제공함으로써 데이터의 불확실성을 효과적으로 다룰 수 있다.

또한, 퍼지 C-Means 군집화는 군집의 중첩을 허용하는 유연성을 가지고 있다. 데이터 포인트가 여러 군집에 동시에 속할 수 있으므로 현실 세계의 복잡한 데이터 구조를 더 정확하게 모델링할 수 있다. 그렇기 때문에 퍼지 C-Means 군집화는 데이터의 불확실성을 다루는 데 강력하다. 명확한 군집 경계가 없는 경우에도 데이터의 다양한 정도로 여러 군집에 속할 수 있다.

마지막으로 퍼지 C-Means 군집화는 이상치에 상대적으로 강한 특성을 가진다. 극단적인 값이 군집 결과에 미치는 영향이 K-Means 보다 덜하다고 볼 수 있다. 이러한 장점들은 퍼지 C-Means 군집화가 데이터의 다양성, 불확실성, 중첩 등에 대한 민감성을 줄여주며, 실제 데이터의 특성을 더 잘 반영할 수 있도록 하기 때문에 퍼지 군집화를 선택했다.

5.2.2 군집 개수 결정

```
# pastel 톤의 색상 10가지 생성
colors_list = sns.color_palette("pastel", 10)

# 리스트로 변환
colors = list(colors_list)

pca_result = pca.fit_transform(pca_df)

# 반복문 설정 및 시각화 준비
fig1, axes1 = plt.subplots(3, 3, figsize=(8, 8))
alldata = pca_result.T # 각 데이터 포인트에 대해 두 행 (x, y)를 얻기 위해 전치
fpcs = []

for ncenters, ax in enumerate(axes1.reshape(-1), 2):
    cntr, u, u0, d, jm, p, fpc = fuzz.cluster.cmeans(
        alldata, ncenters, 2, error=0.005, maxiter=1000, init=None)

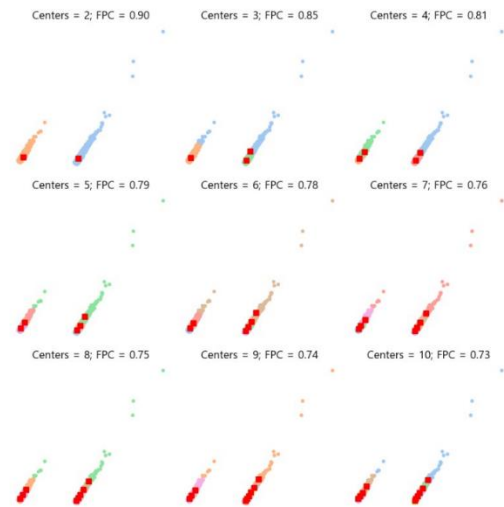
    # fpc값 저장
    fpcs.append(fpc)

    # 할당된 클러스터 플로팅
    cluster_membership = np.argmax(u, axis=0)
    for j in range(ncenters):
        ax.plot(pca_result[cluster_membership == j, 0],
                pca_result[cluster_membership == j, 1], '.', color=colors[j])

    # 군집 별 중심 표시
    for pt in cntr:
        ax.plot(pt[0], pt[1], 'rs')

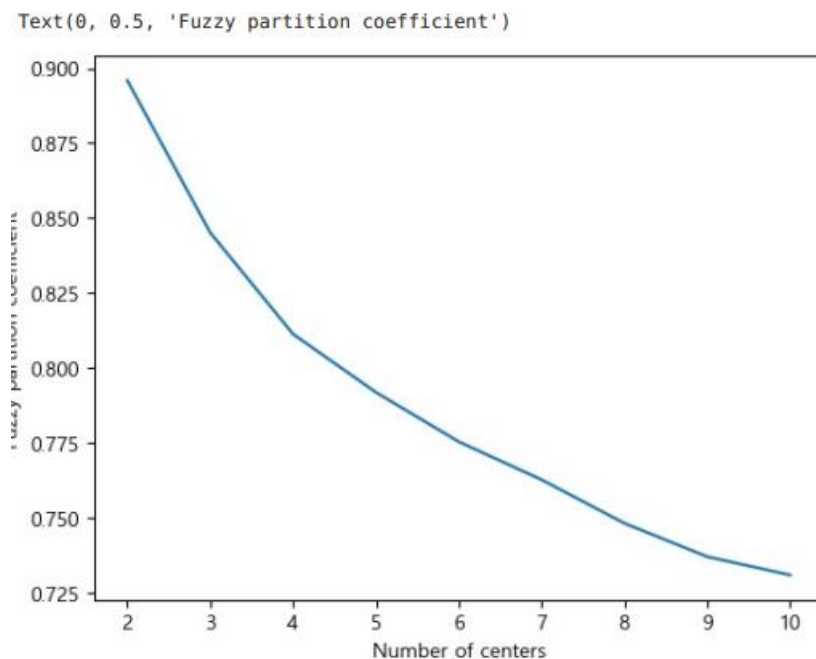
    ax.set_title('Centers = {0}; FPC = {1:.2f}'.format(ncenters, fpc))
    ax.axis('off')

fig1.tight_layout()
```



퍼지 모델의 적절한 군집의 개수를 찾기 위해 모델 시각화를 진행했다. Fpc(퍼지 분할 계수)가 1에 가까울 수록 적합한 군집이라 할 수 있다. 즉 군집의 수가 2 개인 군집화가 가장 최적이지만 군집 수가 너무 적어 오히려 정확도가 떨어질 것이라고 판단하여 차선책으로 군집의 수가 3 개인 군집화를 채택했다.

```
# 적절한 군집 수 찾기
fig2, ax2 = plt.subplots()
ax2.plot(np.r_[2:11], fpcs)
ax2.set_xlabel("Number of centers")
ax2.set_ylabel("Fuzzy partition coefficient")
```



역시 FPC는 군집이 2개일 때 가장 높으며, 첫번째 시각화 자료와 똑같은 결과를 보여주고 있다.

5.2.3 퍼지 모델 생성

```
# 3개 클러스터 센터로 퍼지 모델 재생성
cntr, u_orig, _ = fuzz.cluster.cmeans(
    alldata, 3, 2, error=0.005, maxiter=1000)

# 새로운 색상 리스트 정의
custom_colors = ['skyblue', 'lightcoral', 'lightgreen']

# 시각화
fig2, ax2 = plt.subplots()
ax2.set_title('Fuzzy Clustering Model')

# 각 클러스터에 대해 새로운 색상 사용
for j, color in enumerate(custom_colors):
    ax2.plot(alldata[0, u_orig.argmax(axis=0) == j],
            alldata[1, u_orig.argmax(axis=0) == j], 'o',
            label='series ' + str(j), color=color)

ax2.legend()
plt.show()
```



3 개의 클러스터 센터를 가지고 퍼지 모델을 재생성했다.

```
# Fuzzy C-means 알고리즘을 적용할 함수
def fuzzy_cmeans_clustering(pca_df, num_clusters, max_iter=100, m=2):
    # 데이터 프레임의 값을 배열로 변환
    data_array = pca_df.values.T

    # Fuzzy C-means 알고리즘 적용
    cntr, u, u0, d, jm, p, fpc = fuzz.cluster.cmeans(
        data_array, num_clusters, m, error=0.005, maxiter=max_iter, init=None)

    # 소속도(u) 중에서 가장 큰 값의 인덱스를 군집으로 매핑
    cluster_membership = np.argmax(u, axis=0)

    return cluster_membership
```

```
# 군집 수 설정
num_clusters = 3

# Fuzzy C-means 알고리즘을 이용하여 군집 할당
pca_df['cluster_membership'] = fuzzy_cmeans_clustering(pca_df, num_clusters)

# 결과 출력
print(pca_df)
```

	pc1	pc2	cluster_membership
0	-1.920234	0.139668	0
1	-1.847308	0.735709	0
2	-1.828064	0.846632	0
3	-2.041044	0.021555	0
4	0.944492	-0.692932	2
...	...	...	...
8793	0.880686	-1.472293	2
8794	0.878163	-0.989161	2
8795	1.044120	0.356458	2
8796	0.832373	-1.000031	2
8797	-1.994179	-0.838981	0

pca 데이터를 가지고 만든 퍼지 모델에 클러스터 멤버십을 할당하여 출력했다. pc1 과 pc2 는 각각 첫 번째와 두 번째 주성분을 나타낸다. 예를 들어, 첫 번째 데이터 포인트의 (pc1, pc2) 값은 (-1.920234, 0.139668)이며, 주성분은 각 데이터 포인트를 새로운 축으로 투영한 값으로, 주로 데이터의 분산을 설명하는 데 사용된다. pc1은 pc2에 비해 절댓값이 크기 때문에 pc1 이 각 컬럼에 대한 분산 값을 더 많이 설명한다.

5.2.4 군집 할당 및 파악

```
# cluster membership 컬럼 df에 추가
df['cluster_membership'] = fuzzy_cmeans_clustering(pca_df, num_clusters)
df.head()
```

	id	payAmt	payAmt_average	age	gender_남성	gender_여성	rate	cluster_membership
0	5443	31100.0	36652.333333	50	0	1	0.80	1
1	158486	66243.0	43622.000000	48	0	1	0.95	1
2	148906	61494.0	61494.000000	53	0	1	0.00	1
3	158409	27050.0	27050.000000	31	0	1	0.00	1
4	76233	25000.0	20203.000000	47	1	0	0.00	0

고객 데이터 프레임에 클러스터 멤버십을 추가했다.

```
# 각 클러스터의 통계량 파악
print(df[cluster_membership == 0].describe())
print(df[cluster_membership == 1].describe())
print(df[cluster_membership == 2].describe())
```

```
count    133.000000    1.330000e+02    1.330000e+02    133.000000    133.000000 \
mean    115612.563910    2.494228e+05    1.719513e+05    41.218045    0.969925
std      61402.645207    2.140813e+05    1.123677e+05    8.439929    0.171440
min       826.000000    3.400000e+04    0.000000e+00    21.000000    0.000000
25%      82691.000000    1.596800e+05    1.453030e+05    34.000000    1.000000
50%     128236.000000    1.907600e+05    1.633080e+05    41.000000    1.000000
75%     161161.000000    2.465700e+05    1.957000e+05    47.000000    1.000000
max     238717.000000    2.008700e+06    1.060500e+06    70.000000    1.000000
```

```
gender_여성    rate    cluster_membership
count    133.000000    133.000000    133.0
mean      0.030075    0.221805         2.0
std       0.171440    0.693622         0.0
min       0.000000    0.000000         2.0
25%       0.000000    0.000000         2.0
50%       0.000000    0.000000         2.0
75%       0.000000    0.000000         2.0
max       1.000000    2.900000         2.0
```

```
count    513.000000    513.000000    513.000000    513.000000    513.0 \
mean    132050.561404    87280.172846    46744.634054    39.516569    0.0
std      64844.650406    42069.506497    30023.514677    7.988530    0.0
min       310.000000    14000.000000    0.000000    21.000000    0.0
25%      80171.000000    58100.000000    0.000000    34.000000    0.0
50%     150339.000000    73665.000000    54857.666667    40.000000    0.0
75%     177848.000000    115620.000000    68356.000000    45.000000    0.0
max     238911.000000    219907.000000    125120.000000    66.000000    0.0
```

```
gender_여성    rate    cluster_membership
count    513.0    513.000000    513.0
mean      1.0    0.171832         1.0
std       0.0    0.578843         0.0
min       1.0    0.000000         1.0
25%       1.0    0.000000         1.0
50%       1.0    0.000000         1.0
75%       1.0    0.000000         1.0
max       1.0    3.300000         1.0
```

```
count    318.000000    318.000000    318.000000    318.000000    318.0 \
mean    124268.056604    138876.850629    95608.720718    40.933962    1.0
std      60257.373972    53162.022645    38954.170795    8.259898    0.0
min      113.000000    23600.000000    0.000000    22.000000    1.0
25%     85126.000000    106518.000000    86411.562500    35.000000    1.0
50%     133932.500000    122980.500000    104223.500000    40.500000    1.0
75%     170297.500000    143711.500000    118852.250000    46.750000    1.0
max     238779.000000    347390.000000    180740.000000    66.000000    1.0
```

```
gender_여성    rate    cluster_membership
count    318.0    318.000000    318.0
mean      0.0    0.108962         2.0
std       0.0    0.444692         0.0
min       0.0    0.000000         2.0
25%       0.0    0.000000         2.0
50%       0.0    0.000000         2.0
75%       0.0    0.000000         2.0
max       0.0    2.900000         2.0
```

군집 0 은 평균나이 41.2 세, 평균 월 보험료 171,951.3 원, 대부분이 남성, 평균 건강위험도 0.2 의 집단으로 묶였다. 군집 1 은 평균나이 39.5 세, 여성, 평균 월 보험료 46744.6 원, 평균 건강위험도 0.17 의 집단으로 묶였고, 군집 2 은 평균나이 40.9 세, 남성, 평균 월 보험료 95608.7 원, 평균 건강위험도 0.10 의 집단으로 묶였다.

5.2.5 담보코드와 군집 병합

```
# id와 cluster_membership 컬럼만 뽑아 새로운 데이터프레임 생성
cluster_df = df[['id', 'cluster_membership']]
cluster_df
```

	id	coverageCode	cluster_membership
0	182457	A5999	2
1	182457	A5999	2
2	182457	A4199	2
3	182457	A4199	2
4	182457	A5199	2
...	...	...	...
111922	28	A5200	0
111923	28	A6200	0
111924	28	A9999	0
111925	28	A6100	0
111926	28	A6300	0

클러스터 멤버십을 고객데이터에 추가한다음, 아이디와 클러스터 멤버십 컬럼만 뽑아 새로운 데이터 프레임을 형성한 후 이를 담보코드와 다시 병합하였다.

5.2.6 담보코드 추천

```
# 각 사용자가 속한 군집 결정
user_clusters = fuzzy_cmeans_clustering(pca_df, num_clusters)

# 처음 10개 행 출력
print("First 10 rows:")
for user_id, cluster_id in enumerate(user_clusters[:10]):
    print(f"User {user_id + 1} belongs to Cluster {cluster_id + 1}")

# 끝에서 10개 행 출력
print("\nLast 10 rows:")
for user_id, cluster_id in enumerate(user_clusters[-10:]):
    print(f"User {user_id + 1} belongs to Cluster {cluster_id + 1}")

# 각 사용자에게 대한 추천 수행
for target_user_id in range(len(user_clusters)):
    target_cluster_id = user_clusters[target_user_id]

    # 군집 내 다른 사용자 찾기
    similar_users = np.where(user_clusters == target_cluster_id)[0]

    # 추천 대상 사용자가 아직 평가하지 않은 아이템 찾기
    unrated_items = np.where(pca_df.iloc[target_user_id] == 0)[0]

    # 추천 아이템 선택 (다른 사용자들이 높게 평가한 아이템 중에서)
    recommendations = np.mean(pca_df.iloc[similar_users], axis=0)

    # 아이템의 평균이 0인 경우 (아무도 평가하지 않은 경우) 모든 아이템을 추천
    if np.all(recommendations == 0):
        final_recommendations = unrated_items[:3]
    else:
        # 추천 아이템 선택 (다른 사용자들이 높게 평가한 아이템 중에서)
        recommended_items = np.argsort(recommendations)[-3:-1]
        # 최종 추천
        final_recommendations = [item for item in recommended_items if item in unrated_items][:3]

    # final_recommendations가 비어있지 않은 경우에만 출력
    if final_recommendations:
        print(f"\nTop 5 Recommendations for User {target_user_id + 1}: {final_recommendations}")

    # 상위 5개만 출력하도록 수정
    if target_user_id == 11:
        break
```

First 10 rows:
 User 1 belongs to Cluster 2
 User 2 belongs to Cluster 2
 User 3 belongs to Cluster 2
 User 4 belongs to Cluster 2
 User 5 belongs to Cluster 1
 User 6 belongs to Cluster 3
 User 7 belongs to Cluster 1
 User 8 belongs to Cluster 1
 User 9 belongs to Cluster 1
 User 10 belongs to Cluster 1

Last 10 rows:
 User 1 belongs to Cluster 2
 User 2 belongs to Cluster 1
 User 3 belongs to Cluster 1
 User 4 belongs to Cluster 2
 User 5 belongs to Cluster 1
 User 6 belongs to Cluster 1
 User 7 belongs to Cluster 1
 User 8 belongs to Cluster 1
 User 9 belongs to Cluster 1
 User 10 belongs to Cluster 2

Top 5 Recommendations for User 1: [2]

Top 5 Recommendations for User 2: [2]

Top 5 Recommendations for User 3: [2]

Top 5 Recommendations for User 4: [2]

Top 5 Recommendations for User 12: [2]

```
# 추천된 군집이 갖는 담보코드 리스트 출력
codes = pivot_table.loc[2]
codes_sorted = codes.sort_values(ascending=False)
li = codes_sorted.head(10).index.tolist()
print('cluster 2가 많이 보유하고 있는 담보코드 리스트:', li)
```

cluster 2가 많이 보유하고 있는 담보코드 리스트: ['A5199', 'A5999', 'A4199', 'A4200', 'A4210', 'A1300', 'A4104', 'A9999', 'A5200', 'A3300']

퍼지 클러스터링을 통해 결정된 군집 개수를 가지고 사용자에게 군집을 할당하였고, 실제 사용자가 어떤 군집인지 처음 열개 행, 끝에서 열개 행 출력하여 확인이 가능하다. 각 사용자에 대한 추천을 수행하기 위해 군집 내의 다른 사용자가 아직 갖고 있지 않은 담보코드 군집을 찾아 추천 담보코드 군집을 선택해 출력한다음 추천된 군집이 갖는 담보코드의 리스트를 출력하여 결과를 확인한다.

5. 3. K-Means

5.3.1 K-Means Cluster Model 설명

K-Means 클러스터란 비지도 학습의 한 종류로서, 데이터를 K 개의 클러스터로 군집화하는 알고리즘을 말한다. 이 알고리즘은 각 클러스터의 중심을 찾고, 각 데이터 포인트를 가장 가까운 중심에 할당하여 군집을 형성한다.

K-Means 클러스터 알고리즘은 먼저 임의의 위치에 K 개의 클러스터 중심을 배치하고, 각 데이터 포인트를 가장 가까운 클러스터 중심에 할당한다. 이후 각 클러스터에 할당된 데이터 포인트를 기반으로 새로운 클러스터 중심을 계산하는데, 이는 각 클러스터에 속한 데이터 포인트의 평균 또는 중앙값을 통해 계산한다. 데이터 포인트의 클러스터 할당과 클러스터 중심의 재계산을 반복하며 각 데이터 포인트의 클러스터가 조정되며, 이 과정은 클러스터 내 제곱합(SSE)와 같은 일정 기준에 도달하거나 더 이상 클러스터의 변화가 없을 때까지 반복한다.

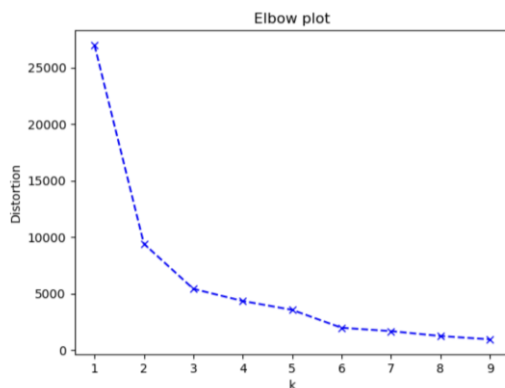
5.3.2 Elbow 기법을 이용한 적절한 K 개의 클러스터 개수 결정

```
# SSE 값 리스트
inertia_arr = []

# 최적의 클러스터 개수 범위
K = range(1, 10)

for k in K:
    # k가 1~10까지의 K-Means 모델링
    k_c_model = KMeans(n_clusters=k, random_state=1024, n_init=1).fit(df_pca)
    inertia = k_c_model.inertia_
    inertia_arr.append(inertia) # 리스트에 해당 K-Means 모델의 SSE값 저장

# Elbow Plot 그래프 그리기
plt.plot(K, inertia_arr, 'bx--')
plt.xlabel('k')
plt.ylabel('Distortion')
plt.title('Elbow plot')
plt.show()
```



K-Means 클러스터는 적절한 K 개의 클러스터 개수를 결정하는 것이 가장 중요하다. 클러스터 개수 결정에 도움을 주는 기법이 Elbow 기법인데, Elbow 기법이란 여러 K 값을 이용하여 K-Means 모델링을 만들어보고 각 K 값에 대해 Inertia를 계산하여 SSE의 변화를 시각적으로 확인할 수 있는 기법을 말한다. K 값이 증가함에 따라 SSE는 자연적으로

감소하는데, 이는 클러스터 개수가 많아질수록 각 데이터 포인트가 자신의 클러스터 중심에 가까워지게 되어 유의미한 군집을 형성할 수 없다. 따라서 Elbow 기법을 통해 팔꿈치(Elbow)처럼 꺾이는 것처럼 SSE 변화가 갑자기 줄어드는 지점을 찾을 수 있고, SSE 변화가 급격히 줄어드는 시점부터의 K는 무의미한 군집 개수로 판단할 수 있도록 도와준다.

상기 코드의 K-Means 모델은 고객의 인류통계학적 데이터 중 나이, 성별, 평균 건강위험도, 평균 월 보험료에 따라 고객 데이터를 군집시킨 모델이다. 그래프를 보면, k가 2일 때 SSE 값이 급격히 줄어드는 것을 확인할 수 있는데, 이는 고객의 성별에 의해 군집이 형성된 영향이라고 판단하였으며, 성별이라는 특성만으로 고객을 군집하기에는 비약적이라고 판단하였다. 따라서 데이터 도메인적 특성을 고려하여 k=2 다음으로 SSE가 급격히 줄어드는 k=6을 k값으로 선정하여 K-Means 클러스터를 모델링하고자 한다.

5.3.3 PCA를 이용하여 클러스터 시각화

```
import matplotlib.markers as markers

# k가 6인 K-Means 클러스터 모델링
kmeans_pca = KMeans(n_clusters=6, random_state=1024, n_init=1).fit(df_pca)

k_df_pca = df_pca.copy()

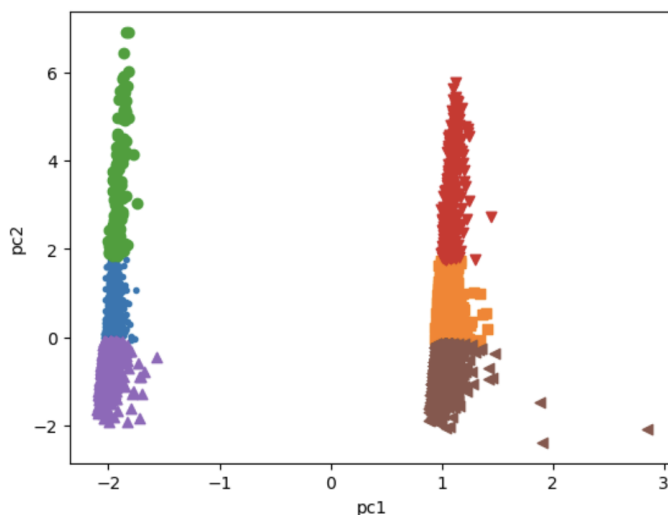
# 각 데이터의 클러스터 아이디 할당
k_df_pca['cluster_id'] = kmeans_pca.labels_

markers = list(markers.MarkerStyle.markers.keys())

# 각 클러스터 아이디에 대한 데이터들의 좌표를 서로 다른 marker로 그래프에 표시
for i, marker in enumerate(markers):
    x_val = k_df_pca[k_df_pca['cluster_id'] == i]['pc1']
    y_val = k_df_pca[k_df_pca['cluster_id'] == i]['pc2']
    plt.scatter(x_val, y_val, marker=marker)

plt.xlabel('pc1')
plt.ylabel('pc2')

Text(0, 0.5, 'pc2')
```



고객 데이터 중 나이, 성별, 평균 건강위험도, 평균 월 보험료를 이용하여 6개의 클러스터 개수를 가지는 K-Means 클러스터를 모델링하였다. 하지만 4가지 특성에 따르는 클러스터 모델을 시각화하기 위해서는 4차원 그래프가 필요하며, 이는 구현하는 것에 제약이 있기 때문에, PCA를 이용하여 2차원으로 축소시킨 후 K-Means 클러스터를 모델링하여 시각화 하였다.

상기 그래프를 보면, 군집이 크게 2가지로 형성된 것을 확인할 수 있으며, 이는 성별 특성에 의한 것으로 판단된다. 또한, 각 성별에 따르는 군집 내에서도 각각 3개의 군집이 형성된 것을 확인할 수 있다.

5.3.4 K-Means 클러스터(k=6) 모델의 군집 별 의미 파악

```
: # K가 6인 K-Means 모델링
# PCA 데이터가 아닌 표준정규화한 데이터들을 이용하여 모델링
kmeans = KMeans(n_clusters=6, random_state=1024, n_init=1).fit(X_scaled)

: cluster_df = df.copy() # cluster_df라는 데이터프레임에 기존 데이터프레임 복사
cluster_df['cluster_id'] = kmeans.labels_ # cluster_df의 cluster_id라는 컬럼에 각 데이터에 따르는 클러스터 아이디 매핑
```

군집별 데이터 의미 분석

```
: # 각 컬럼별 분석을 위해 고객 고유 키인 id 컬럼을 제거하여 t_df 데이터프레임에 저장
t_df = cluster_df.drop(columns=['id'])
# 각 클러스터 아이디 별 평균 값 출력
t_df.groupby('cluster_id').mean()
```

```
:
      payAmt      age  gender_남성  gender_여성      rate
cluster_id
0  52100.973473  48.823616    0.000000    1.000000    0.019815
1  47345.507237  35.519685    1.000000    0.000000    0.008017
2  53968.734489  43.889423    0.728365    0.271635    1.924880
3  196048.895394  40.599156    0.932489    0.067511    0.020253
4  52684.954766  34.750514    0.000000    1.000000    0.010864
5  48054.305410  49.523913    1.000000    0.000000    0.010565
```

상기 코드는 K가 6개인 K-Means 클러스터 모델을 통해 군집 형성된 6개의 클러스터의 의미 및 특성을 파악한 코드이다. 클러스터 별 의미를 분석하기에 적절한 지표로 각 컬럼의 평균값을 선정하였으며, 연속적인 값 위주로 표현되는 컬럼이 다수임에 따라 컬럼의 평균값이 적절하다고 판단하였다.

0번 클러스터는 평균 52,101 원의 월 평균 보험료를 지출하며, 평균 49세 여성이며, 평균 0.0198의 평균 건강위험도를 가지고 있는 군집이다.

1번 클러스터는 평균 47,346 원의 월 평균 보험료를 지출하며, 평균 36세 남성이며, 평균 0.008의 평균 건강위험도를 가지고 있는 군집이다.

2번 클러스터는 평균 53,969 원의 월 평균 보험료를 지출하며, 평균 44세이며, 남성이 주로 분포해 있고, 평균 1.9249의 평균 건강위험도를 가지고 있는 군집이다.

3번 클러스터는 평균 196,049 원의 월 평균 보험료를 지출하며, 평균 41세이며, 남성이 주로 분포해 있고, 평균 0.0202의 평균 건강위험도를 가지고 있는 군집이다.

4번 클러스터는 평균 52,685 원의 월 평균 보험료를 지출하며, 평균 35세 여성이며, 평균 0.0109의 평균 건강위험도를 가지고 있는 군집이다.

5번 클러스터는 평균 48,054 원의 월 평균 보험료를 지출하며, 평균 50세 남성이며, 평균 0.0106의 평균 건강위험도를 가지고 있는 군집이다.

5.3.5 고객 군집과 보장담보코드 간의 User-based CF Pivot Table

```
cluster_df = cluster_df[['id', 'cluster_id']] # cluster_df에서 고객의 id와 클러스터 id만 추출

# df_4(보장담보코드 데이터프레임)와 cluster_df를 고객의 id를 기준으로 조인 병합
merged_df = df_4.merge(cluster_df[['id', 'cluster_id']], on='id')
# 병합한 데이터프레임을 각 클러스터 아이디 별로 그룹화하고, 각 그룹에 속한 보장담보코드의 개수를 계산하여 pivot table을 형성
pivot_table = merged_df.groupby('cluster_id')['coverageCode'].value_counts().unstack(fill_value=0)
pivot_table
```

coverageCode	A1100	A1101	A1199	A1200	A1299	A1300	A1301	A1399	A1400	A1401	...	A9607	A9608	A9609	A9611	A9615	A9616	A9617	A9619	A...
cluster_id																				
0	229	21	41	129	9	671	106	24	94	24	...	5	8	8	7	0	1	370	261	
1	601	10	40	187	21	1404	96	50	146	60	...	25	17	12	16	1	2	715	500	
2	97	7	10	42	2	290	52	10	31	10	...	4	2	1	5	1	0	158	120	
3	80	2	1	5	0	153	13	0	7	0	...	10	1	4	1	0	0	110	86	
4	329	10	28	101	19	784	65	31	67	27	...	4	2	4	5	0	5	472	347	
5	419	29	44	201	6	1104	220	39	134	38	...	22	17	12	7	2	0	648	408	

6 rows x 139 columns

K-Means 클러스터 모델에 의해 매핑된 각 고객의 클러스터 아이디를 담고 있는 데이터와 각 고객이 보유하는 보장담보코드를 담고 있는 데이터를 고객 id 를 기준으로 조인 병합하면, 각 고객의 클러스터 아이디와 보유 보장담보코드를 담고 있는 데이터가 만들어진다. 만들어진 데이터 프레임에 클러스터 아이디를 기준으로 그룹화하고, 그룹화하며 각 고객이 보유하고 있는 보장담보코드의 개수를 sum 계산한다. 이렇게 그룹화함으로써 각 군집 별 보유하는 보장담보코드들의 개수를 담고 있는 pivot table 을 구성할 수 있다.

5.3.6. 새로운 고객의 군집 예측과 해당 군집의 인기 보장담보코드 추천

```
: age = int(input("고객의 나이를 입력하세요: "))
gender = input("고객의 성별을 입력하세요 (남성은 'M', 여성은 'F'): ")
avg_rate = float(input("고객의 평균 건강위험도를 입력하세요: "))
avg_payAmt = float(input("고객의 평균 월보험료를 입력하세요: "))

if gender == 'M':
    gender_M = 1
    gender_F = 0
elif gender == 'F':
    gender_M = 0
    gender_F = 1

new_customer = pd.DataFrame([
    {'payAmt': avg_payAmt, 'age': age, 'gender_남성': gender_M, 'gender_여성': gender_F, 'rate': avg_rate}
])

# 새로운 고객 데이터 정규화 및 PCA
new_customer_scaled = X_scaler.transform(new_customer)
new_customer_pca = pca.transform(new_customer_scaled)
pca_cols = ['pc1', 'pc2']
new_customer_pca = pd.DataFrame(data=new_customer_pca, columns=pca_cols)

# 새로운 고객 데이터 군집 예측
predicted_cluster = kmeans_pca.predict(new_customer_pca)
print('고객님의 군집 아이디: ', predicted_cluster)

# pivot table의 새로운 고객 군집의 담보코드 리스트
coverages = pivot_table.loc[int(predicted_cluster)]
coverages_sorted = coverages.sort_values(ascending=False)
recommend_coverages = coverages_sorted.head(5).index.tolist()
print('고객님과 유사한 고객들이 많이 가입한 담보코드: ', recommend_coverages)

고객의 나이를 입력하세요: 20
고객의 성별을 입력하세요 (남성은 'M', 여성은 'F'): F
고객의 평균 건강위험도를 입력하세요: 1.5
고객의 평균 월보험료를 입력하세요: 40000
고객님의 군집 아이디: [0]
고객님과 유사한 고객들이 많이 가입한 담보코드: ['A5199', 'A5999', 'A4200', 'A4199', 'A4210']
```

상기 코드는 위에서 구성한 pivot table 을 이용하여 새로운 고객의 군집을 예측하고, 해당 군집의 인기 보장담보코드 상위 5 개를 출력하는 코드이다. 새로운 고객의 정보를 입력 받고, 입력 받은 데이터를 데이터프레임에 담아 표준정규화 및 PCA 를 적용한 후, PCA 가 적용된 K-Means 모델에 대입함으로써 새로운 고객의 클러스터를 예측할 수 있다. 이후 pivot table 을 통해 예측된 클러스터에 해당하는 보장담보코드 개수 데이터에서 내림차순으로 정렬하여 상위 5 개를 보여줌으로써 해당 클러스터가 가장 많이 보유하고 있는 보장담보코드를 출력할 수 있다.

상기 출력물의 시나리오, 평균 건강위험도가 1.5 인 20 세 여성이 월 평균 40,000 원의 보험료를 지출하고 있을 때, 해당 고객의 군집은 0 번 클러스터(평균 52,101 원의 월 평균 보험료를 지출하며, 평균 49 세 여성이며, 평균 0.0198 의 평균 건강위험도를 가지고 있는 군집)이며, 0 번 클러스터가 가장 많이 보유하고 있는 보장담보코드 상위 5 개인 A5199, A5999, A4200, A4199, A4210 보장담보코드를 추천하고 있다.

6. 각 모델링 최종 결과 해석 및 개선점 탐구

6. 1 DBSCAN

고객의 인류통계학적 데이터인 나이, 성별과 고객의 건강정보 데이터 중 암종류별 건강 위험도의 평균 위험도, 고객별 평균 월보험료, 총 4 가지 컬럼을 이용하여 DBSCAN 군집화를 수행하였다. 그러기 위해서 첫번째로는 데이터셋을 로드하고 전처리하는 과정을 수행했다. 이 과정에는 불필요한 컬럼 삭제, 나이 계산, 원-핫 인코딩 적용 등이 포함된다. 그리고 df_1 과 df_2 를 병합하여 새로운 데이터프레임 df 를 생성하고, 중복된 id 를 제거했다. 성별 컬럼에 대해서는 원-핫 인코딩을 적용했다. 그리고 표준화된 데이터에 DBSCAN 클러스터링을 적용했다. 그리고 클러스터링 결과를 바탕으로 각 클러스터별 선호하는 담보 코드를 분석했다. 그리고 클러스터의 개수는 DBSCAN 의 eps 를 통해 조절 하였고 한눈에 어떤 클러스터가 메인 클러스터인지 볼 수 있게 시각화 하였다. 그리고 고객의 정보를 입력받으면 선호 담보 코드를 출력할 수 있게 하였다.

이 결과를 해석해보자면 DBSCAN 결과는 각 클러스터별로 선호되는 담보 코드가 무엇인지 파악할 수 있게 해주는 데 그 이유는 클러스터링을 통해 유사한 특성을 가진 데이터 포인트들을 그룹화하고, 각 그룹이 선호하는 담보 코드를 분석하기 때문이다. 이러한 분류 모델링은 보험 기관이 고객의 니즈를 더 잘 이해하고 적절한 보험 상품을 제안하는 데 도움이 될 수 있다고 생각된다.

6. 2 FUZZY

고객의 인류통계학적 데이터인 나이, 성별과 고객의 건강정보 데이터 중 암종류별 건강 위험도의 평균 위험도, 고객별 평균 월보험료, 총 4 가지 컬럼을 이용하여 퍼지 군집화를 진행하였다. 데이터의 전처리로는, 성별 데이터의 경우 범주형 데이터이기에 원핫인코딩 기법을 이용하여 전처리하였으며, 나이와, 성별(원핫인코딩 0/1), 평균 건강위험도, 평균 월보험료 데이터들 간의 단위가 모두 다르기에 Standard Scaler 를 이용하여 표준정규화하였다.

user_clusters 에 저장된 클러스터 군집을 이용해 처음 10 개 사용자와 마지막 10 개 사용자에게 대한 군집 결과가 출력했다. 전체 데이터 셋 중 처음 10 명의 사용자의 경우, 대부분이 'Cluster 1'에 속해 있고, 마지막 10 명의 사용자의 경우, 'Cluster 1'과 'Cluster 2'에 속한 사용자들이 혼재되어 있다. 각 사용자에게 대한 추천 결과를 보면, 타겟 군집(cluster membership = 2)에 해당하는 사용자들 목록을 위에서부터 5 명 출력했다. 추천된 군집이 갖는 담보코드 리스트를 출력했으며, A5199, A5999, A4199, A4200, A4210, A1300, A4104, A9999, A5200, A3300 의 순으로 추천한 것을 확인할 수 있다.

Fuzzy 모델은 수치를 입력 받아 새로운 고객 데이터를 생성하여 담보코드를 추천하지 못했지만, 모델을 개선하여 이러한 점까지 가능하다면 세부적이고 정확도 높은 모델링이 될 것으로 보인다.

6. 3 K-Means

고객의 인류통계학적 데이터인 나이, 성별과 고객의 건강정보 데이터 중 암종류별 건강 위험도의 평균 위험도, 고객별 평균 월보험료, 총 4 가지 컬럼을 이용하여 K-Means 군집화를 진행하였다. 데이터의 전처리로는, 성별 데이터의 경우 범주형 데이터이기에 원핫인코딩 기법을 이용하여 전처리하였으며, 나이와, 성별(원핫인코딩 0/1), 평균 건강위험도, 평균 월보험료 데이터들 간의 단위가 모두 다르기에 Standard Scaler 를 이용하여 표준정규화하였다.

K-Means 군집화 모델에서 가장 중요한 k 선정에 도움을 얻기 위해 elbow 기법을 활용해본 결과, k 가 2 일때 기울기가 많이 꺾여 있는 것을 알 수 있었다. 다만, 데이터와 도메인의 특성 상 k 가 2 일 경우, 고객의 성별만으로 군집될 가능성이 크기 때문에 2 다음으로 SSE 가 급격히 변화된 6 을 k 로 선정하였다.

총 6 개의 클러스터를 학습한 모델을 시각화하기 위해 PCA 를 이용한 차원 축소를 하였으며, 2 차원으로 축소된 모델의 산점도를 보면 양극단으로 데이터가 위치해 있는 것을 알 수 있다. 이는 성별이라는 데이터 특성에 의한 것이라고 생각되며, 성별로 나뉘어진 군집 속에서 나이와 평균 건강위험도, 평균 월보험료 데이터의 특성 대로 군집화되는 것으로 추측할 수 있다.

K-Means 모델을 통해 얻은 클러스터 정보들을 각 고객별로 매핑한 후, 매핑한 클러스터 데이터셋과 담보코드 데이터셋을 병합 및 그룹화하여 각 군집별 보유하고 있는 담보코드의 개수를 표현하는 pivot_table 을 생성하였다. pivot_table 을 통해 각 군집별 선호하는 담보코드를 확인할 수 있다. 현 모델은 고객 데이터의 수가 부족하고 고객을 설명할 수 있는 특성이 적어서 만족스러운 군집이 되지 않지만, 많은 데이터와 고객을 보다 잘 설명할 수 있는 컬럼들을 추가한다면 보다 나은 모델링이 될 것으로 보인다.

결론적으로 새로운 고객의 데이터의 클러스터를 K-Means 모델을 통해 예측하고, pivot_table 에서 새로운 고객의 클러스터에 해당하는 담보코드들 중 카운트가 높은 순으로 고객의 추천을 해줌으로써 담보코드 영업의 타당성과 신뢰성을 향상시킬 수 있을 것으로 생각된다.